ASSIGNMENT 3

Elevator Control System Using Bridge and Adapter Patterns

Student: Nurzhan bakyt

Assignment: Assignment 3 - Bridge and Adapter Patterns

Abstract

The current report deals with the description of the implementation of the Elevator Control System using both Bridge and Adapter design patterns in one architectural solution. The proposed approach allows for the support of different types of motor controllers. In particular, the Bridge design pattern helps in separating the request types for elevators from the motor controller implementations, while the Adapter design pattern is used in order to work with the existing analog motor controller which does not implement the controller interface.

1. Introduction

Modern object-oriented systems have to be developed taking into consideration the possibility of creating software without excessive dependencies between the elements of the system. Design patterns allow the developers to create the extensible and reusable architecture of the system.

The goal of this assignment is to design and develop the system that utilizes the Bridge and Adapter design patterns. The chosen domain for the project is the elevator control system. The elevator system accepts the transportation requests for both passengers and freights and uses several types of the motor controllers.

The presented solution shows the combination of two design patterns and their application to a specific problem in software engineering. The Bridge design pattern provides the separation of the business logic from the hardware implementations while the Adapter allows using the legacy motor controller without changing the source code.

2. Problem Description

The motor system used in elevator systems depends on the environment where the elevators have been installed. New elevator installations usually use digital controllers while industrial elevators usually use hydraulic controllers. In some older buildings, analog motors could have been installed using various interfaces and communication methods.

The software must support all of these technologies at the same time while offering a uniform API to submit the requests. The software must be extensible to add new controllers and new requests without modifying any existing code.The system therefore requires:

- independent request hierarchy;
- independent controller hierarchy;

- support for a legacy analog controller;
- runtime controller selection;
- unified exception handling.

3. Requirements Analysis

As specified in the assignment specifications, the system should meet the following criteria:

- Having an abstraction hierarchy with at least two refined abstractions.
- Implementor interface.
- At least three implementations of the implementor.
- One of the implementations should be converted from a legacy class that is not compatible.
- The implementation should handle all failures using a common exception.
- Dynamic selection of implementations.
- UML documentation.
- Junit 5 unit tests.The designed system satisfies all of these requirements.

4. System Architecture Overview

The proposed solution comprises two separate hierarchies linked together by using the Bridge design pattern.

The first hierarchy describes the hierarchy of elevator requests:

- ElevatorRequest
- PassengerRequest
- FreightRequest

The second hierarchy describes the motor implementations:

- MotorController
- DigitalMotorController
- HydraulicMotorController
- AnalogMotorAdapter

A legacy component called AnalogMotorLegacyUnit is included in the design using the Adapter design pattern.

The component which selects controller implementations at run-time is the registry component.

5. Bridge Pattern Implementation

5.1 Purpose of the Bridge Pattern

Bridge enables independent development of abstraction and implementation hierarchies.

Without Bridge, each different pair of request and controller types would have needed its own class.

Examples:

- PassengerDigitalController
- PassengerHydraulicController
- PassengerAnalogController
- FreightDigitalController
- FreightHydraulicController
- FreightAnalogController

With Bridge, the number of classes will not grow exponentially with the size of the system.

5.2 Abstraction

The abstract class ElevatorRequest is the Abstraction component of the Bridge Design Pattern.

Roles:

- keep reference to MotorController;
- specify elevator request processing logic;
- make current floor data accessible.

The abstraction does not have any dependencies on concrete motor components.

5.3 Refined Abstractions

PassengerRequest

PassengerRequest is a normal passenger transport.

Duties:

- to move the elevator to the requested floor;
- to leave doors open after arriving.

This class implements passenger behavior regardless of the controller implementation.

FreightRequest

FreightRequest represents cargo transportation.

Responsibilities:

- validate cargo weight;
- prevent overload situations;
- delegate movement to the selected MotorController.

This implementation adds business rules for freight transportation.

5.4 Implementor Interface

MotorController represents the Implementor side of the Bridge.

The interface defines two operations:

moveToFloor(int floor)

getCurrentFloor()

Any controller technology can participate in the architecture by implementing this interface.

5.5 Concrete Implementors

DigitalMotorController

DigitalMotorController represents a modern elevator controller.

Features:

- floor range checking;
- direct floor movement;
- exception-based error handling.

HydraulicMotorController

HydraulicMotorController simulates hydraulic elevator technology.

Features:

- movement distance limitations;
- pressure constraints;
- exception-based error reporting.

AnalogMotorAdapter

Although AnalogMotorAdapter appears as a controller implementation, it is actually an Adapter around a legacy component.

It allows legacy hardware to participate in the Bridge architecture.

6. Adapter Pattern Implementation

6.1 Legacy Component

The class AnalogMotorLegacyUnit represents an existing legacy controller that cannot directly implement MotorController.

Several incompatibilities exist.

Method incompatibility

The legacy component provides:

setVoltageLevel()

readVoltageLevel()

instead of:

moveToFloor()

getCurrentFloor()

Data incompatibility

The legacy controller works with voltage levels rather than floor numbers.

Error handling incompatibility

Modern controllers throw exceptions, while the legacy controller returns integer status codes.

Examples:

- STATUS_OK
- STATUS_OVERLOAD
- STATUS_STALLED
- STATUS_OUT_OF_RANGE

Therefore, direct integration is impossible.

6.2 Adapter Responsibilities

AnalogMotorAdapter performs the following tasks:

- converts floors to voltage levels;
- converts voltage levels back to floors;
- translates status codes into exceptions;
- implements MotorController.

This allows the legacy component to be used transparently.

6.3 Failure Translation

Legacy Status	MotorException Error Type
STATUS_OK	No Exception
STATUS_OVERLOAD	OVERLOAD
STATUS_STALLED	STALLED
STATUS_OUT_OF_RANGE	OUT_OF_RANGE
Unknown Value	UNKNOWN

This translation ensures a consistent error-handling mechanism across the entire system.

7. Dynamic Implementor Selection

Complexity module implementation was necessary according to the task description. The selected approach for this is Dynamic Implementor Selection.

In the class MotorControllerRegistry, the controllers' type information is read from the configuration file, and then the corresponding implementation is created.

Example of the configuration:

car-A=digital

car-B=hydraulic

car-C=analog-legacy

The client code doesn’t know which particular implementation of the controller is used. The client just asks for the controller in the registry.

8. UML Class Diagram

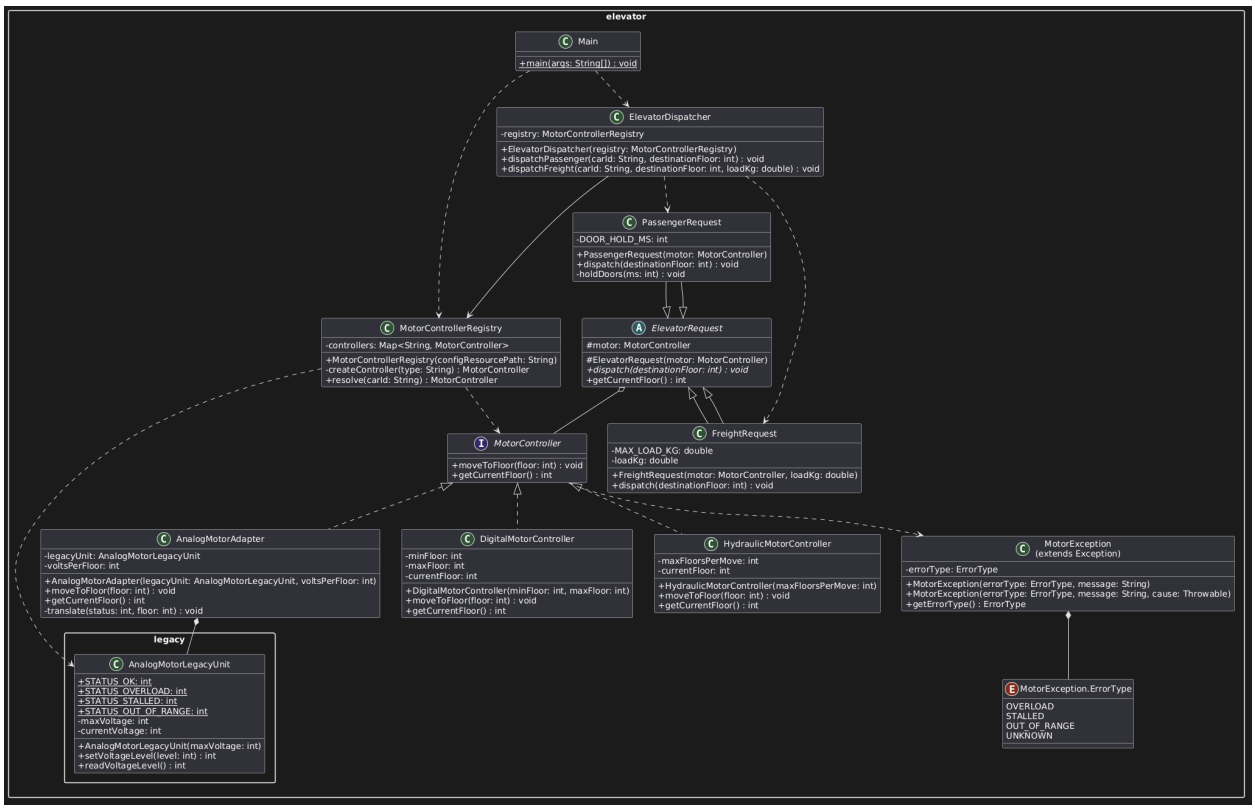


Figure 1. UML Class Diagram of the Elevator Control System

The above UML diagram presents the overall architecture of the software design. It displays the Bridge association between ElevatorRequest and MotorController, the inheritance structure of different requests, and the Adapter association between AnalogMotorAdapter and AnalogMotorLegacyUnit.

9. Unit Testing

JUnit 5 and Mockito were used to validate the implementation.

9.1 PassengerRequestTest

The test verifies:

- correct delegation to MotorController;
- proper exception propagation.

9.2 FreightRequestTest

The test verifies:

- delegation to controller;
- overload detection;
- prevention of invalid movement operations.

9.3 AnalogMotorAdapterTest

The test verifies:

- successful adaptation;
- overload translation;
- stalled translation;
- out-of-range translation;
- unknown-status translation;
- floor-to-voltage conversion.

9.4 MotorControllerRegistryTest

The test verifies:

- runtime selection of implementations;
- proper handling of invalid identifiers.

10. Advantages and Limitations

Advantages

The architecture provides several benefits:

- Separation of abstraction and implementation.
- Easy integration of new controller technologies.
- Compliance with the Open/Closed Principle.
- Runtime controller selection.
- Legacy system compatibility.
- Improved maintainability and scalability.

Limitation

The adapter relies on a fixed voltage-per-floor conversion factor.

Future hardware with different communication protocols may require additional adapters.

11. Conclusion

This assignment successfully implements both the Bridge and the Adapter patterns in order to resolve an actual software engineering problem. Using the Bridge pattern enables the separation of the elevator request processing hierarchy from the hardware hierarchy; the two may evolve independently of one another.

Implementing the Adapter pattern facilitates the implementation of an absolutely incompatible legacy controller without changing the code. The overall design fulfills all the requirements of the assignment: it allows for the dynamic selection of the implementor,

provides for unified exception handling, is properly documented in terms of UML diagrams, and includes proper unit testing.

Github: Nbst7/SDP-Assignment3